DOKUMENTACJA TECHNICZNA IMPLEMENTACJI FRONTENDU

Moduł Dewelopera D: Oszczędności, Inwestycje, Kalkulator Emerytalny

Projekt	FinApp - Osobisty Asystent Finansowy	**Data utworzenia**	26 maja 2026 r.
Autor	Deweloper D (Frontend Lead)	**Status**	Zaakceptowany / Wdrożony do main
Technologia	Flutter (Dart), Riverpod	**Wersja modułu**	v1.0.0-MVP

1. Podsumowanie Menedżerskie (Executive Summary)

Niniejszy dokument stanowi oficjalne podsumowanie prac inżynieryjnych wykonanych przez **Dewelopera D** w ramach implementacji warstwy wizualnej oraz logicznej aplikacji **FinApp**. Zakres prac obejmował pełną realizację trzech kluczowych komponentów funkcjonalnych aplikacji: modułu kont oszczędnościowych, modułu kont inwestycyjnych oraz interaktywnego kalkulatora emerytalnego.

Architektura rozwiązania została zaprojektowana w oparciu o paradygmat stanowy **Riverpod**, gwarantując pełną separację logiki biznesowej od interfejsu użytkownika (UI). Rozwiązanie pomyślnie przeszło testy integracyjne z lokalną symulacją API i zostało scalone z gałęzią główną (`main`), stanowiąc stabilną bazę pod rozwój pozostałych składowych systemu (w tym modułów przypisanych Deweloperowi C).

2. Architektura Systemu i Zarządzanie Stanem

W celu zapewnienia maksymalnej wydajności, reaktywności oraz testowalności kodu, wdrożono zaawansowane zarządzanie stanem przy użyciu biblioteki `flutter_riverpod`. Zamiast podstawowych, statycznych struktur typu `FutureProvider`, zastosowano nowoczesną architekturę opartą na klasach rozszerzających `AsyncNotifier`.

> **Zaleta Architektoniczna AsyncNotifierProvider**
>
> Wdrożona struktura pozwala na asynchroniczne ładowanie danych początkowych (stan `loading`/`data`/`error`) przy jednoczesnym wystawieniu metod mutujących stan (np. dodawanie, usuwanie), eliminując potrzebę ręcznego zarządzania flagami zajętości interfejsu.

Dane wewnątrz aplikacji reprezentowane są przez spójny model danych `AssetModel`, który mapuje strukturę rekordów bazy danych dostarczoną w dokumentacji schematów backendowych przez Dewelopera B.

3. Moduł Oszczędności (SavingsScreen)

3.1. Charakterystyka Funkcjonalna

Moduł umożliwia użytkownikowi kompleksowe zarządzanie poduszką finansową oraz rachunkami oszczędnościowymi. Widok prezentuje dynamicznie generowany wykres podziału aktywów oraz szczegółową listę kafelków reprezentujących poszczególne konta.

3.2. Komponenty Techniczne i Implementacja

- **Wykres Kołowy (Donut Chart):** Zaimplementowany przy użyciu pakietu `fl_chart`. Wykres w czasie rzeczywistym oblicza proporcje poszczególnych kont na podstawie zunifikowanej wartości bazowej.
- **Wielowalutowość i Normalizacja Danych:** Zgodnie z dobrymi praktykami inżynierii finansowej, wykres nie sumuje bezpośrednio nominalnych wartości różnych walut. Wdrożono wewnętrzny mechanizm walidacji i przeliczania walut obcych (USD, EUR, GBP) na walutę bazową (PLN) według zdefiniowanych bezpiecznych przeliczników:
  - 1 USD = 4.00 PLN
  - 1 EUR = 4.30 PLN
  - 1 GBP = 5.05 PLN
- **Formatowanie Prezentacji Liczb:** Napisano autorski parser oparty na wyrażeniach regularnych (Regex), formatujący wyświetlane kwoty poprzez separację tysięcy przecinkami (np. `1,250,000 PLN`), co dramatycznie podnosi czytelność interfejsu użytkownika.

4. Moduł Inwestycji (InvestmentsScreen)

Moduł dedykowany do śledzenia rachunków maklerskich oraz dedykowanych kont emerytalnych (IKE, IKZE). W ramach tego modułu zastosowano wzorzec **Reużywalności Kodu (Code Reuse)**.

Zamiast powielać strukturę formularza wprowadzania danych, uniwersalny ekran `AddAssetScreen` został wyposażony w flagi konstrukcyjne (`isInvestment`). Formularz automatycznie adaptuje swoje etykiety, nagłówki oraz – co kluczowe – kieruje strumień danych do właściwego zarządcy stanu (`investmentsProvider` zamiast `savingsProvider`).

5. Kalkulator Emerytalny (RetirementScreen)

Zgodnie z ustaleniami z Deweloperem B, kalkulator emerytalny działa całkowicie po stronie klienta (Client-Side Computing), nie obciążając serwera zapytaniami sieciowymi. Zmiana parametrów na suwakach wywołuje natychmiastowe przeliczenie prognozy finansowej.

5.1. Logika Matematyczna (Silnik Obliczeniowy)

Silnik symulacji opiera się na klasycznym wzorze finansowym na **wartość przyszłą renty z dołu (Future Value of an Ordinary Annuity)**, uwzględniającym regularne, comiesięczne wpłaty kapitału podlegające kapitalizacji miesięcznej:

$$FV = PMT \times ((1 + r)^n - 1) / r$$

Gdzie poszczególne parametry definiowane są jako:

- FV (Future Value) – Końcowy szacowany kapitał zgromadzony w wieku emerytalnym (65 lat).
- PMT (Payment) – Wysokość regularnej składki miesięcznej (ustawiana suwakiem `_monthlyContribution`).
- r – Miesięczna stopa zwrotu, wyliczana z rocznej stopy procentowej: $r = (Roczna\ Stopa / 100) / 12$.
- n – Całkowita liczba okresów (miesięcy) oszczędzania, wyliczana jako: $n = (65 - Obecny\ Wiek) \times 12$.

> **Obsługa Przypadków Brzegowych (Edge Case Protection)**
> W scenariuszu, w którym użytkownik definiuje stopę zwrotu na poziomie dokładnie 0%, algorytm automatycznie przełącza się na liniowe mnożenie składek ($FV = PMT \times n$), zapobiegając krytycznemu błędowi dzielenia przez zero.

6. Integracja z Backendem i Zgodność z Dokumentacją MVP

Analiza plików dokumentacji dostarczonych przez Dewelopera B wykazała częściowe przesunięcie wymagań funkcjonalnych dla wersji MVP. Interfejs frontendowy został precyzyjnie dostosowany do możliwości technicznych backendu:

Operacja CRUD	Status na Serwerze (Dev B)	Implementacja we Flutterze (Dev D)	Działanie końcowe
Create (Zapis)	Zaimplementowane (POST)	Pełna obsługa formularza	Dane przesyłane do Providera
Read (Odczyt)	Zaimplementowane (GET)	Asynchroniczne listy i wykres	Automatyczne odświeżanie UI
Update (Edycja)	**Brak (Wyciszone w MVP)**	Kod przygotowany, ukryty w UI	Opcja wyłączona w widokach
Delete (Usuwanie)	Zaimplementowane (DELETE)	Ikonka kosza (Action Button)	Kaskadowe usuwanie z bazy

7. Repozytorium i Przepływ Pracy (Git Workflow)

Prace deweloperskie były prowadzone w sposób izolowany, zgodnie z najlepszymi praktykami GitFlow:

1. Utworzono dedykowaną gałąź funkcjonalną: `feature/add-asset-form`.
2. Po zakończeniu implementacji i pomyślnej weryfikacji działania wielowalutowości, formatowania liczb oraz wykresów, dokonano konsolidacji kodu.

3. Zmiany zostały pomyślnie scalone (merged) z główną gałęzią produkcyjną `main`.

```
$ git status
$ git add .
$git commit -m "Feat: Zakończenie prac nad MVP dla Dev D (Inwestycje, Kalkulator, Wykres)"$ 
$git merge feature/add-asset-form$ git push origin main
```

[Koniec Dokumentu - Wersja Cyfrowa PDF]